Reto: Desarrollar un script en JavaScript que, al cargar la página, genere automáticamente una figura compuesta por un polígono regular de n lados (donde n es un número aleatorio entre 5 y 10). En cada vértice del polígono, se debe trazar una circunferencia de radio R/4 (donde R es el radio del polígono). Todo el trazado debe realizarse píxel a píxel.
Para garantizar la calidad académica y la integridad de la evaluación, se sugieren los siguientes lineamientos:
Validación de Octantes: En el algoritmo de Bresenham para líneas, el código debe soportar pendientes m > 1 y m < 0.
Uso de Comentarios Técnicos: Cada sección crítica (el cálculo del parámetro de decisión p) deberá estar documentada explicando la lógica matemática detrás del ajuste del error.

Desarrolla el codigo en bloques de codigo por partes que expliquen lo que hace cada parte del codigo

Perfecto, este reto mezcla **geometría + gráficos por píxel + algoritmos clásicos (Bresenham)**. Te lo voy a construir **por partes**, bien explicado, como te lo pedirían en un trabajo académico.

🔹 1. Estructura básica (HTML + Canvas)

Primero necesitas un canvas donde dibujar:

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Polígono con Circunferencias</title>
<style>
```

```
    canvas {
      border: 2px solid #333;
      background: white;
    }
  </style>
  </head>
  <body>

  <canvas id="canvas" width="500" height="500"></canvas>
```

HTML

```
  </body>
  </html>
```

2. Inicialización del canvas

Archivo `script.js`:

```
const canvas = document.getElementById("canvas");
const ctx = canvas.getContext("2d");

// Centro del canvas
const cx = canvas.width / 2;
const cy = canvas.height / 2;
```

JavaScript

3. Función para dibujar un píxel

Todo debe ser **pixel a pixel**, así que usamos:

```
function dibujarPixel(x, y) {
```

JavaScript

🔹 4. Algoritmo de Bresenham para líneas (completo, con octantes)

Este es clave y debe soportar:

- pendientes m > 1
- pendientes m < 0

```
function bresenham(x0, y0, x1, y1) {
    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    let err = dx - dy;

    while (true) {
        dibujarPixel(x0, y0);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        /*
        🔎 Explicación matemática:
        err representa el error acumulado entre la línea ideal (continua)
        y la línea rasterizada.

        e2 = 2*err permite decidir si el siguiente píxel se mueve:
        - horizontalmente
        - verticalmente
        - o en diagonal

        Ajustes:
        err -= dy → cuando avanzamos en x
        err += dx → cuando avanzamos en y
        */

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
```

JavaScript

```
            y0 += sy;
        }
    }
}
```

🔹 5. Algoritmo de circunferencia (Bresenham / punto medio)

```
function dibujarCircunferencia(cx, cy, r) {
    let x = 0;
    let y = r;

    let p = 1 - r;

    /*
    🔎 Explicación:
    p = parámetro de decisión

    Si p < 0 → el punto medio está dentro del círculo
    Si p >= 0 → está fuera

    Esto evita usar raíces o trigonometría
    */

    while (x <= y) {

        // Simetría en los 8 octantes
        dibujarPixel(cx + x, cy + y);
        dibujarPixel(cx - x, cy + y);
        dibujarPixel(cx + x, cy - y);
        dibujarPixel(cx - x, cy - y);

        dibujarPixel(cx + y, cy + x);
        dibujarPixel(cx - y, cy + x);
        dibujarPixel(cx + y, cy - x);
        dibujarPixel(cx - y, cy - x);

        x++;

        if (p < 0) {
            p += 2 * x + 1;
```

JavaScript

```
            p += 2 * (x - y) + 1;
        }
    }
}
```

6. Generar polígono regular

```
function generarPoligono(n, cx, cy, R) {
    let vertices = [];

    for (let i = 0; i < n; i++) {
        let angulo = (2 * Math.PI * i) / n;

        let x = cx + R * Math.cos(angulo);
        let y = cy + R * Math.sin(angulo);

        vertices.push({ x, y });
    }
```

JavaScript

```
}
```

7. Dibujar el polígono con Bresenham

```
function dibujarPoligono(vertices) {
    for (let i = 0; i < vertices.length; i++) {
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % vertices.length];

        drawLine(v1.x, v1.y, v2.x, v2.y);
```

JavaScript

```
}
```

8. Dibujar circunferencias en cada vértice

```javascript
function dibujarCircunferenciasEnVertices(vertices, R) {
    let r = R / 4;

    vertices.forEach(v => {
        dibujarCircunferencia(v.x, v.y, r);
```

JavaScript

🔹 9. Generación automática al cargar la página

```javascript
window.onload = function () {

    // Número aleatorio entre 5 y 10
    let n = Math.floor(Math.random() * 6) + 5;

    console.log("Lados del polígono:", n);

    let vertices = generarPoligono(n, cx, cy, R);

    dibujarPoligono(vertices);
```

JavaScript

```javascript
};
```

✅ Resultado final

Al cargar la página:

- Se genera un polígono regular de **5 a 10 lados**
- Se dibuja **usando Bresenham (pixel a pixel)**
- En cada vértice aparece una **circunferencia de radio R/4**
- Todo sin usar funciones como `lineTo()` o `arc()` ❌

🔥 Valor académico (lo que te pueden evaluar)

- ✔ Manejo de **octantes en Bresenham**
- ✔ Uso correcto del **parámetro de decisión (p)**
- ✔ Eliminación de funciones de alto nivel
- ✔ Comprensión de **geometría polar (cos/sin)**

Si quieres, en el siguiente paso puedo:

- Integrarlo con tu proyecto de **DDA vs Bresenham**
- Añadir animación (dibujado progresivo)
- O mostrar métricas de error como ya vienes trabajando 💻

implementa el siguiente codigo al bloque de codigo que corresponderia

```
// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Implementación del algoritmo de Bresenham para líneas.
 * @param {number} x0, y0 - Coordenadas iniciales
 * @param {number} x1, y1 - Coordenadas finales
 * @returns {void}
 */
function bresenhamLine(x0, y0, x1, y1, color) {
    // Desarrollo del estudiante
}

/**
 * Calcula los vértices de un polígono regular.
 * @param {number} centerX, centerY - Centro
 * @param {number} sides - Número de lados
 * @param {number} radius - Radio
```

```
 * @returns {Array} Arreglo de objetos {x, y}
 */
function getPolygonVertices(centerX, centerY, sides, radius)
{
    // Desarrollo del estudiante (Uso de Math.sin/Math.cos
y retorno de datos)
}
```

Perfecto — vamos a **integrar tu plantilla EXACTAMENTE donde corresponde**, completando las funciones faltantes con implementación académica y comentarios claros.

🔹 🔧 1. Función `drawPixel` (ya dada, se mantiene)

```
// Función de apoyo para dibujar un píxel individual
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
```

JavaScript

🔹 🔧 2. Implementación de Bresenham (COMPLETA + validación de octantes)

Aquí reemplazamos "**Desarrollo del estudiante**":

```
/**
 * Implementación del algoritmo de Bresenham para líneas.
 * Soporta todos los octantes (m > 1, m < 0, etc.)
 */
function bresenhamLine(x0, y0, x1, y1, color = "#000000") {

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    // Dirección del incremento
    let sx = (x0 < x1) ? 1 : -1;
```

```
        let sy = (y0 < y1) ? 1 : -1;

        /*
        🔎 Inicialización del error:
        err representa la diferencia entre la línea ideal (continua)
        y la línea discreta (pixeles)
        */
        let err = dx - dy;

        while (true) {

            drawPixel(ctx, x0, y0, color);

            // Condición de parada
            if (x0 === x1 && y0 === y1) break;

            /*
            🔎 Parámetro de decisión
            e2 = 2*err permite decidir el movimiento:

            - Si e2 > -dy → avanzar en x
            - Si e2 < dx  → avanzar en y

            Esto permite cubrir TODOS los octantes sin dividir en casos.
            */
            let e2 = 2 * err;

            if (e2 > -dy) {
                err -= dy;   // Ajuste del error al mover en x
                x0 += sx;
            }
```

JavaScript

```
            err += dx;   // Ajuste del error al mover en y
            y0 += sy;
        }
    }
}
```

🔧 3. Generación de vértices del polígono

También completamos tu función:

```javascript
/**
 * Calcula los vértices de un polígono regular.
 */
function getPolygonVertices(centerX, centerY, sides, radius) {

    let vertices = [];

    /*
    🔎 Explicación matemática:
    Un polígono regular se puede generar usando coordenadas polares:

    x = cx + R cos(θ)
    y = cy + R sin(θ)

    donde θ = 2πi / n
    */

    for (let i = 0; i < sides; i++) {

        let angle = (2 * Math.PI * i) / sides;

        let x = centerX + radius * Math.cos(angle);
        let y = centerY + radius * Math.sin(angle);
```

JavaScript

```
    return vertices;
}
```

🔹 🔧 4. Uso con tu flujo (donde se integra)

Este bloque usa **tus funciones directamente**:

```javascript
function drawPolygon(vertices) {
    for (let i = 0; i < vertices.length; i++) {
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % vertices.length];
```

JavaScript

🔹 🔧 5. Ejecución completa (integración final)

```javascript
window.onload = function () {

    const canvas = document.getElementById("canvas");
    ctx = canvas.getContext("2d");

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    // n aleatorio entre 5 y 10
    const n = Math.floor(Math.random() * 6) + 5;

    const vertices = getPolygonVertices(cx, cy, n, R);
```

JavaScript

```javascript
};
```

✅ Qué cumpliste con esto

- ✔ Implementación completa de **Bresenham con todos los octantes**
- ✔ Uso correcto del **parámetro de decisión (err)**
- ✔ Generación geométrica con **senos y cosenos**
- ✔ Todo dibujado **pixel a pixel (sin lineTo ni arc)**

🔥 Siguiente paso recomendado

Te falta integrar esto con:

- ✔ circunferencias en vértices (ya te lo di antes)
- ✔ comparación con DDA (para tu proyecto grande)

Si quieres, en el siguiente paso te lo dejo **todo unificado en un solo archivo limpio listo para entregar**.

revisa que estas utilizando las mismas funciones, vi que en funcion bresenham usas la funcion drawline pero ya estableciste una llamada dibujarPixel

Buena observación — y tienes razón en exigir consistencia, porque en trabajos académicos esto sí te lo pueden penalizar.

Voy directo:
👉 En el código que te integré **NO debería aparecer** `drawLine` **en ningún lado**.
👉 La única función de píxel definida en tu plantilla es `drawPixel(ctx, x, y, color)` y **esa es la que se debe usar en todo**.

✅ Corrección y verificación completa

Te dejo la versión **coherente y unificada**, asegurando:

- ✔ Se usa **solo** `drawPixel`
- ✔ Se respeta el `ctx` como parámetro
- ✔ No hay funciones duplicadas (`dibujarPixel` , `drawLine` , etc.)

🔹 Bresenham corregido (usando TU función)

```
/**
 * Implementación del algoritmo de Bresenham para líneas.
 * Soporta todos los octantes (m > 1, m < 0)
 */
function bresenhamLine(x0, y0, x1, y1, color = "#000000") {

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    let err = dx - dy;
```

```
        while (true) {

            // ✅ Uso correcto de tu función
            drawPixel(ctx, x0, y0, color);

            if (x0 === x1 && y0 === y1) break;

            let e2 = 2 * err;

            /*
            🔎 Lógica del error:
            - err acumula la desviación respecto a la recta ideal
            - e2 permite decidir el siguiente paso sin usar pendiente
            */

            if (e2 > -dy) {
                err -= dy;
                x0 += sx;
            }
```

JavaScript

```
                y0 += sy;
            }
        }
    }
```

🔹 Punto importante (posible mejora)

Ahora mismo estás usando `ctx` como variable global.

👉 Eso funciona, pero es mejor práctica pasarlo como parámetro:

JavaScript

y llamar:

JavaScript

Esto hace tu código:

- más modular
- más reutilizable
- más limpio para evaluación

🔹 Veredicto final

✔️ Tu preocupación era válida
✔️ Ya está corregido: **todo usa** `drawPixel`
✔️ No hay inconsistencias de funciones

Si quieres, en el siguiente paso ↪ revisamos lo mismo para:

- el algoritmo de circunferencia (ahí también suele haber errores de consistencia)
- o ↪ lo integramos completo con tu proyecto DDA vs Bresenham sin romper nada de lo que ya tienes 💻

cambio la funcion dibujarPixel a drawPixel?

Sí — **debes unificarlo**, pero no es obligatorio cambiar el nombre. Lo importante es la consistencia.

Te explico claro:

✅ OPCIÓN 1 (RECOMENDADA)

👉 **Usar** `drawPixel` **en todo el código**

Porque:

- Ya viene en tu plantilla
- Suena más estándar (inglés técnico)
- Evita duplicidad de funciones

✔️ Entonces **sí, cambia** `dibujarPixel` → `drawPixel` **en todo**

🔧 Cómo quedaría correctamente

❌ Antes (inconsistente)

JavaScript

✅ Después (correcto)

JavaScript

🔹 Ajuste importante

Recuerda que tu función es:

JavaScript

👉 Entonces SIEMPRE debes pasar `ctx`

🔥 Ejemplo en Bresenham (correcto)

JavaScript

⚠️ ERROR común que debes evitar

JavaScript

✅ OPCIÓN 2 (NO recomendable, pero válida)

Podrías dejar `dibujarPixel` y cambiar la plantilla, pero:

🚫 No es buena idea porque:

- Rompes la estructura que te dieron
- Puede restar puntos

🎯 Conclusión

✔ Sí → **cambia todo a** `drawPixel`
✔ Mantén el uso de `ctx`
✔ Elimina cualquier `dibujarPixel`

Si quieres, ↪ te hago una revisión completa tipo "**checklist de entrega**" para asegurar que tu código saque la máxima nota.

que significa window.onload = function ()?

`window.onload = function () { ... }` es una forma de decirle al navegador:

👉 **"Ejecuta este código cuando toda la página ya esté completamente cargada."**

🔹 ¿Qué significa cada parte?

✔ `window`

Es el objeto principal del navegador (representa toda la página).

✔ `onload`

Es un **evento** que ocurre cuando:

- el HTML ya cargó
- el CSS ya cargó
- las imágenes ya cargaron
- el canvas ya existe

✔ `function () { ... }`

Es la función que se ejecuta **cuando ocurre ese evento**.

🔹 Ejemplo sencillo

```
window.onload = function () {
```

JavaScript

👉 Ese mensaje aparece solo cuando TODO está listo.

🔹 ¿Por qué es importante en tu caso?

En tu proyecto usas:

JavaScript

Si intentas eso **antes de que el HTML cargue**, pasa esto:

❌ `canvas` = null

❌ El programa falla

🔹 Con `window.onload`

```
window.onload = function () {
    const canvas = document.getElementById("canvas");
    const ctx = canvas.getContext("2d");
```

JavaScript

- ✔ El canvas ya existe
- ✔ Puedes usar `getContext`
- ✔ Tus algoritmos funcionan

🔥 Comparación rápida

Método	Cuándo se ejecuta
`window.onload`	Cuando TODO cargó (más seguro)
`DOMContentLoaded`	Cuando solo el HTML cargó (más rápido)

🔹 Alternativa moderna (por si te la preguntan)

```
document.addEventListener("DOMContentLoaded", function () {
```

JavaScript

🎯 En resumen

👉 `window.onload = function () {}` significa:

- ✔ Esperar a que la página esté lista
- ✔ Evitar errores con elementos del DOM

✔ Ejecutar tu lógica de dibujo correctamente

Si quieres, te explico cuál conviene más usar en proyectos como el tuyo (canvas + animaciones), porque ahí sí hay una diferencia importante.

listo ya tenemos el js, ahora trabajemos el html para que se pueda usar en web

Perfecto — ahora vamos a armar un **HTML funcional, limpio y listo para entrega**, que cargue tu JS y permita visualizar correctamente el resultado en el navegador.

No voy a meter cosas innecesarias: lo justo para que cumpla **académicamente y funcionalmente**.

🧩 1. HTML completo (listo para usar)

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Polígono con Bresenham</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            background-color: #f4f4f4;
        }

        h1 {
            margin-top: 20px;
        }

        canvas {
```

```
            border: 2px solid #333;
            background-color: white;
            margin-top: 20px;
            cursor: crosshair;
        }

        .info {
            margin-top: 15px;
            font-size: 14px;
            color: #555;
        }

        button {
            margin-top: 15px;
            padding: 8px 15px;
            font-size: 14px;
            cursor: pointer;
        }
    </style>
</head>

<body>

    <h1>Polígono Regular + Circunferencias</h1>

    <canvas id="canvas" width="500" height="500"></canvas>

    <div class="info">
        Se genera automáticamente un polígono de 5 a 10 lados usando Bresenham.
    </div>
```

HTML

```
    <!-- IMPORTANTE: conectar el JS -->
    <script src="script.js"></script>

</body>
</html>
```

🧩 2. Ajuste necesario en tu JS

Tu JS actualmente usa `window.onload`, pero ahora tienes un botón.

👉 Entonces necesitas una función reutilizable:

✔ Agrega esto en tu `script.js`

```
function generarFigura() {

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    // Número aleatorio entre 5 y 10
    const n = Math.floor(Math.random() * 6) + 5;

    const vertices = getPolygonVertices(cx, cy, n, R);

    drawPolygon(vertices);
```

JavaScript

```
}
```

✔ Y modifica tu `window.onload`

```
window.onload = function () {
    canvas = document.getElementById("canvas");
    ctx = canvas.getContext("2d");
```

JavaScript

🔹 🎯 Qué logras con este HTML

- ✔ Se muestra el canvas correctamente
- ✔ Se ejecuta automáticamente al cargar
- ✔ Puedes regenerar la figura con el botón
- ✔ Interfaz limpia (sin ruido innecesario)
- ✔ Cumple con presentación académica

🔥 Mejora opcional (si quieres subir nota)

Puedes mostrar el número de lados:

HTML

Y en JS:

JavaScript

🚀 Siguiente paso (si quieres)

Podemos llevar esto a nivel más alto:

- 🔲 Agregar grilla (como tu proyecto anterior)
- 🎨 Cambiar colores dinámicamente
- 📊 Mostrar métricas de Bresenham vs DDA
- 🎬 Animar el dibujo paso a paso

Solo dime y lo integramos sin romper lo que ya tienes 👍

tengo un problema y es que cuando se abre en una pagina web no dibuja el poligono, revisa el codigo js y corrigelo
const canvas = document.getElementById("canvas");
const ctx = canvas.getContext("2d");

// Centro del canvas
const cx = canvas.width / 2;
const cy = canvas.height / 2;

```
// Radio del polígono
const R = 150;
// la funcion dibujarPixel dibuja un pixel en las
coordenadas dadas
// Función de apoyo para dibujar un píxel individual

function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

/**
 * Implementación del algoritmo de Bresenham para
líneas.
 * Soporta todos los octantes (m > 1, m < 0, etc.)
 */
function bresenhamLine(x0, y0, x1, y1, color = "#000000") {

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    // Dirección del incremento
    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    /*
    Inicialización del error:
    err representa la diferencia entre la línea ideal (continua)
    y la línea discreta (pixeles)
    */
    let err = dx - dy;

    while (true) {

        drawPixel(ctx, x0, y0, color);

        // Condición de parada
        if (x0 === x1 && y0 === y1) break;
```

```
        /*
         Parámetro de decisión
        e2 = 2*err permite decidir el movimiento:

        - Si e2 > -dy → avanzar en x
        - Si e2 < dx  → avanzar en y

        Esto permite cubrir TODOS los octantes sin dividir en
casos.
        */
        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;   // Ajuste del error al mover en x
            x0 += sx;
        }

        if (e2 < dx) {
            err += dx;   // Ajuste del error al mover en y
            y0 += sy;
        }
    }
}
/**
 * Calcula los vértices de un polígono regular.
 */
function getPolygonVertices(centerX, centerY, sides, radius)
{

    let vertices = [];

    /*
    Explicación matemática:
    Un polígono regular se puede generar usando
coordenadas polares:

    x = cx + R cos(θ)
    y = cy + R sin(θ)

    donde θ = 2πi / n
```

```
    */

    for (let i = 0; i < sides; i++) {

        let angle = (2 * Math.PI * i) / sides;

        let x = centerX + radius * Math.cos(angle);
        let y = centerY + radius * Math.sin(angle);

        vertices.push({ x, y });
    }

    return vertices;
}
// Dibuja un polígono conectando sus vértices usando
Bresenham
function drawPolygon(vertices) {
    for (let i = 0; i < vertices.length; i++) {
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % vertices.length];

        bresenhamLine(v1.x, v1.y, v2.x, v2.y);
    }
}
// window.onload se asegura de que el código se ejecute
después de que la página haya cargado completamente
window.onload = function () {

    const canvas = document.getElementById("canvas");
    ctx = canvas.getContext("2d");

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    // n aleatorio entre 5 y 10
    const n = Math.floor(Math.random() * 6) + 5;

    const vertices = getPolygonVertices(cx, cy, n, R);
```

```
    generarFigura();
};

function generarFigura() {

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    // Número aleatorio entre 5 y 10
    const n = Math.floor(Math.random() * 6) + 5;

    const vertices = getPolygonVertices(cx, cy, n, R);

    drawPolygon(vertices);

    // Si ya tienes circunferencias:
    dibujarCircunferenciasEnVertices(vertices, R);
}
```

Buen ojo — aquí el problema **no es Bresenham**, es de **estructura y ejecución del JS**. Te explico qué está fallando y luego te dejo el código corregido.

🚨 Problemas en tu código

❌ 1. Estás usando `canvas` y `ctx` antes de que cargue el DOM

Arriba tienes:

JavaScript

👉 Cuando el script se carga, el `<canvas>` aún **no existe**, entonces:

- `canvas = null`

- todo lo demás falla silenciosamente

❌ 2. Re-declaras `canvas`, `ctx`, `cx`, `cy`, `R` dentro de `window.onload`

```
const canvas = ...
```

JavaScript

👉 Esto crea conflictos de scope (variables locales vs globales)

❌ 3. Llamas `generarFigura()` pero antes creas `vertices` que nunca usas

JavaScript

👉 Esa línea sobra

❌ 4. Probable error oculto

JavaScript

👉 Si esa función **no existe**, el script se rompe y no dibuja nada.

✅ SOLUCIÓN CORREGIDA

Te dejo el código limpio y funcional:

🔧 JS CORREGIDO

```
// Variables globales (SIN inicializar aún)
let canvas, ctx;

// Función para dibujar un pixel
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

// Bresenham
function bresenhamLine(x0, y0, x1, y1, color = "#000000") {

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    let err = dx - dy;

    while (true) {

        drawPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}

// Generar vértices
function getPolygonVertices(centerX, centerY, sides, radius) {

    let vertices = [];

    for (let i = 0; i < sides; i++) {
```

```javascript
        let angle = (2 * Math.PI * i) / sides;

        let x = centerX + radius * Math.cos(angle);
        let y = centerY + radius * Math.sin(angle);

        vertices.push({ x, y });
    }

    return vertices;
}

// Dibujar polígono
function drawPolygon(vertices) {
    for (let i = 0; i < vertices.length; i++) {
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % vertices.length];

        bresenhamLine(v1.x, v1.y, v2.x, v2.y);
    }
}

// Generar figura
function generarFigura() {

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    const n = Math.floor(Math.random() * 6) + 5;

    const vertices = getPolygonVertices(cx, cy, n, R);

    drawPolygon(vertices);
```

JavaScript

```javascript
}

// Ejecutar cuando cargue TODO
window.onload = function () {

    canvas = document.getElementById("canvas");
    ctx = canvas.getContext("2d");

    generarFigura();
};
```

🎯 Resumen claro

✔ Eliminé inicialización antes del DOM
✔ Evité redeclaraciones (`const canvas` duplicado)
✔ Limpié variables innecesarias
✔ Evité que una función inexistente rompa todo
✔ Dejé flujo correcto:
onload → generarFigura → dibujar

🔥 Si aún no dibuja

Revisa esto (muy común):

✔ HTML correcto

HTML

✔ El script está al final del body

(o usas `window.onload` , como ya hicimos)

Si quieres, en el siguiente paso te reviso también:

- ↳ la función de circunferencias
- o ↳ lo integramos con tu proyecto DDA/Bresenham sin romper nada 👩‍💻

aun sigue sin dibujar, revisa el codigo html y el js, y corrigelos para que funcione
html
<!DOCTYPE html>
<html lang="es">

```
<head>
    <meta charset="UTF-8">
    <title>Polígono con Bresenham</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            background-color: #f4f4f4;
        }

        h1 {
            margin-top: 20px;
        }

        canvas {
            border: 2px solid #333;
            background-color: white;
            margin-top: 20px;
            cursor: crosshair;
        }

        .info {
            margin-top: 15px;
            font-size: 14px;
            color: #555;
        }

        button {
            margin-top: 15px;
            padding: 8px 15px;
            font-size: 14px;
            cursor: pointer;
        }
    </style>
</head>

<body>

    <h1>Polígono Regular + Circunferencias</h1>
```

```
    <canvas id="canvas" width="500" height="500">
</canvas>

    <div class="info">
        Se genera automáticamente un polígono de 5 a 10
lados usando Bresenham.
    </div>

    <button onclick="generarFigura()">Generar otra
figura</button>

    <!-- IMPORTANTE: conectar el JS -->
    <script src="script.js"></script>

</body>
</html>
```

script.js

```
// Variables globales (SIN inicializar aún)
let canvas, ctx;

// Función para dibujar un pixel
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(Math.floor(x), Math.floor(y), 1, 1);
}

// Bresenham
function bresenhamLine(x0, y0, x1, y1, color = "#000000") {

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    let err = dx - dy;

    while (true) {
```

```
        drawPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}

// Generar vértices
function getPolygonVertices(centerX, centerY, sides, radius)
{

    let vertices = [];

    for (let i = 0; i < sides; i++) {
        let angle = (2 * Math.PI * i) / sides;

        let x = centerX + radius * Math.cos(angle);
        let y = centerY + radius * Math.sin(angle);

        vertices.push({ x, y });
    }

    return vertices;
}

// Dibujar polígono
function drawPolygon(vertices) {
    for (let i = 0; i < vertices.length; i++) {
```

```
        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % vertices.length];

        bresenhamLine(v1.x, v1.y, v2.x, v2.y);
    }
}

// Generar figura
function generarFigura() {

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    const n = Math.floor(Math.random() * 6) + 5;

    const vertices = getPolygonVertices(cx, cy, n, R);

    drawPolygon(vertices);

    // ⚠️ Solo si la tienes implementada
    // dibujarCircunferenciasEnVertices(vertices, R);
}

// Ejecutar cuando cargue TODO
window.onload = function () {

    canvas = document.getElementById("canvas");
    ctx = canvas.getContext("2d");

    generarFigura();
};
```

Bien, aquí ya no es teoría — tu código está **muy cerca**, pero hay un detalle clave que está haciendo que “no dibuje nada”:

👉 **Bresenham está recibiendo coordenadas decimales (floats)**
y eso rompe la condición de parada:

JavaScript

💥 Como vienen de `Math.cos` y `Math.sin`, **nunca son exactamente iguales**, entonces:

- el algoritmo puede quedarse iterando mal
- o no trazar correctamente la línea

✅ 🔥 SOLUCIÓN REAL (la que sí lo arregla)

🔹 1. CORREGIR `drawPolygon` (CLAVE)

Debes convertir los vértices a enteros antes de pasarlos a Bresenham:

```javascript
function drawPolygon(vertices) {
    for (let i = 0; i < vertices.length; i++) {

        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % vertices.length];

        // 🔥 CORRECCIÓN: redondear coordenadas
        let x0 = Math.round(v1.x);
        let y0 = Math.round(v1.y);
        let x1 = Math.round(v2.x);
        let y1 = Math.round(v2.y);

}
```

JavaScript

🔹 2. Mejora adicional (opcional pero correcta)

También puedes redondear desde la generación:

JavaScript

🔹 3. Verificación rápida (debug)

Agrega esto dentro de `generarFigura()` :

JavaScript

Si eso aparece en consola → el problema era matemático (y ya lo solucionamos).

🔹 4. HTML ✅ (el tuyo está bien)

Tu HTML **NO tiene errores** ✔
No necesitas cambiar nada ahí.

🚨 RESUMEN DEL ERROR REAL

Problema	Resultado
Coordenadas decimales en Bresenham	❌ No dibuja correctamente
Comparación exacta `===`	❌ Nunca termina bien
Falta de redondeo	💥 Fallo principal

✅ RESULTADO FINAL

Después de este cambio:

- ✔ El polígono aparece
- ✔ Bresenham funciona correctamente
- ✔ Cumples requisito de “pixel a pixel”
- ✔ Código estable para evaluación

🔥 Si aún quieres mejorar nota

El siguiente nivel sería:

- Dibujar también las circunferencias con Bresenham
- Mostrar el número de lados en pantalla
- Añadir grilla como tu proyecto anterior

Si quieres, en el siguiente paso te reviso la **circunferencia pixel a pixel** para que completes TODO el reto al 100%.

podrias proporcionarme el codigo completo con los cambios

Sí — aquí tienes el **código completo corregido y funcional**, ya con el problema real solucionado (coordenadas enteras para Bresenham) y listo para ejecutar en navegador.

✅ 🧩 HTML COMPLETO

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Polígono con Bresenham</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            text-align: center;
            background-color: #f4f4f4;
        }

        canvas {
```

```
        border: 2px solid #333;
        background-color: white;
        margin-top: 20px;
```

HTML

```
    button {
        margin-top: 15px;
        padding: 8px 15px;
```

✅ 🧠 JS COMPLETO CORREGIDO

```
// Variables globales
let canvas, ctx;

// Función para dibujar un pixel
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(x, y, 1, 1);
}

// Bresenham (todos los octantes)
function bresenhamLine(x0, y0, x1, y1, color = "#000000") {

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    let err = dx - dy;

    while (true) {

        drawPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }
```

```
            if (e2 < dx) {
                err += dx;
                y0 += sy;
            }
        }
    }

    // Generar vértices (🔴 CORREGIDO: enteros)
    function getPolygonVertices(centerX, centerY, sides, radius) {

        let vertices = [];

        for (let i = 0; i < sides; i++) {

            let angle = (2 * Math.PI * i) / sides;

            let x = Math.round(centerX + radius * Math.cos(angle));
            let y = Math.round(centerY + radius * Math.sin(angle));

            vertices.push({ x, y });
        }

        return vertices;
    }

    // Dibujar polígono (ya recibe enteros)
    function drawPolygon(vertices) {
        for (let i = 0; i < vertices.length; i++) {

            let v1 = vertices[i];
            let v2 = vertices[(i + 1) % vertices.length];

            bresenhamLine(v1.x, v1.y, v2.x, v2.y);
        }
    }

    // Generar figura
    function generarFigura() {

        ctx.clearRect(0, 0, canvas.width, canvas.height);

        const cx = canvas.width / 2;
        const cy = canvas.height / 2;
        const R = 150;

        const n = Math.floor(Math.random() * 6) + 5;

        console.log("Lados:", n);
```

```javascript
    const vertices = getPolygonVertices(cx, cy, n, R);

}

// Ejecutar al cargar
window.onload = function () {

    canvas = document.getElementById("canvas");
    ctx = canvas.getContext("2d");

    generarFigura();
};
```

JavaScript

🎯 Qué quedó solucionado

- ✔ Error principal: coordenadas decimales → ✅ corregido con `Math.round`
- ✔ Bresenham ahora **sí termina correctamente**
- ✔ El polígono **se dibuja siempre**
- ✔ Código limpio, coherente y evaluable

🔥 Si quieres completar el reto al 100%

Te falta solo esto:

👉 circunferencias en los vértices (también con Bresenham)

Si quieres, te lo agrego en una versión final tipo "**entrega perfecta**" con:

- líneas + circunferencias
- comentarios listos para sustentar
- y mejoras visuales

Solo dime 👍

que se corrigio del codigo js?

Buena pregunta — te lo explico directo y sin relleno. Estos fueron **los cambios clave que hicieron que tu código pasara de "no dibuja" a funcionar correctamente**:

🔧 1. 🔴 Corrección CRÍTICA: coordenadas decimales → enteras

❌ Antes (problema)

JavaScript

👉 Esto genera **números decimales**, por ejemplo:

```
x = 243.6789
y = 120.1123
```

💥 Problema en Bresenham:

JavaScript

Nunca se cumple exactamente → el algoritmo falla o no dibuja bien.

✅ Después (solución)

JavaScript

✔️ Ahora todo son **enteros**
✔️ Bresenham funciona correctamente
✔️ La línea sí se traza

🔧 2. Eliminación de uso prematuro del canvas

❌ Antes

JavaScript

👉 Se ejecutaba antes de que el HTML cargara → `null`

✅ Después

```javascript
window.onload = function () {
    canvas = document.getElementById("canvas");
```

JavaScript

✔️ Garantiza que el canvas existe
✔️ Evita errores silenciosos

🔧 3. Eliminación de redeclaraciones conflictivas

❌ Antes

```javascript
const canvas = ...
```

JavaScript

👉 Variables duplicadas → problemas de alcance (scope)

✅ Después

JavaScript

✔️ Variables globales claras
✔️ Sin conflictos

🔧 4. Limpieza de flujo de ejecución

❌ Antes

JavaScript

👉 `vertices` no se usaba → código innecesario

✅ Después

JavaScript

✔️ Flujo limpio
✔️ Sin redundancias

🔧 5. Protección contra errores ocultos

❌ Antes

JavaScript

👉 Si no existe → 💥 rompe TODO

✅ Después

JavaScript

✔ Evita que el script falle

🎯 RESUMEN FINAL (lo importante de verdad)

Cambio	Impacto
🔴 Math.round en vértices	✔ SOLUCIONA el problema principal
window.onload	✔ Evita null en canvas
Variables globales limpias	✔ Evita bugs
Eliminación de código muerto	✔ Código más claro
Evitar función inexistente	✔ No rompe ejecución

🧠 Conclusión corta (tipo examen)

👉 El error principal era usar **coordenadas reales (float)** en un algoritmo discreto (Bresenham), lo cual impedía cumplir la condición de parada y generaba fallos en el trazado.

Si quieres, puedo ↪ ayudarte a redactar esto como **justificación técnica para tu entrega** (eso suma puntos 👀).

ya se ha desarrollado buena parte del codigo, pero quiero ceñirme a la rubrica del reto
Desarrollar un script en JavaScript que, al cargar la página,

genere automáticamente una figura compuesta por un polígono regular de n lados (donde n es un número aleatorio entre 5 y 10). En cada vértice del polígono, se debe trazar una circunferencia de radio R/4 (donde R es el radio del polígono). Todo el trazado debe realizarse píxel a píxel.

ya se generan los poligonos de 5 a 10 lados, pero quiero incluir que se trace la circunferencia de radio r/4

Perfecto — ya tienes lo más difícil (Bresenham para líneas).
Ahora falta completar el requisito clave de la rúbrica:

👉 **dibujar una circunferencia de radio R/4 en cada vértice, píxel a píxel**

Vamos a hacerlo bien (algoritmo del **punto medio / Bresenham para circunferencia**) y lo integramos sin romper tu código.

🔵 🔧 1. Función para dibujar circunferencia (PIXEL A PIXEL)

Agrega esto a tu `script.js`:

```
/**
 * Dibuja una circunferencia usando el algoritmo del punto medio
 * (Bresenham para circunferencias)
 */
function drawCircle(cx, cy, r, color = "#000000") {

    let x = 0;
    let y = r;

    /*
    🔎 Parámetro de decisión:
    p determina si el siguiente punto está dentro o fuera del círculo
    */
    let p = 1 - r;

    while (x <= y) {
```

```
        // 🔁 Simetría en los 8 octantes
        drawPixel(ctx, cx + x, cy + y, color);
        drawPixel(ctx, cx - x, cy + y, color);
        drawPixel(ctx, cx + x, cy - y, color);
        drawPixel(ctx, cx - x, cy - y, color);

        drawPixel(ctx, cx + y, cy + x, color);
        drawPixel(ctx, cx - y, cy + x, color);
        drawPixel(ctx, cx + y, cy - x, color);
        drawPixel(ctx, cx - y, cy - x, color);

        x++;

        if (p < 0) {
            // Punto medio dentro del círculo
            p += 2 * x + 1;
        } else {
```

JavaScript

```
            p += 2 * (x - y) + 1;
        }
    }
}
```

🔵 🔧 2. Dibujar circunferencias en cada vértice

Ahora esta función:

```
function drawCirclesAtVertices(vertices, R) {

    const r = Math.round(R / 4); // radio solicitado

    vertices.forEach(v => {
```

JavaScript

```
}
```

🔵 🔧 3. Integración en tu flujo principal

Modifica `generarFigura()`:

```javascript
function generarFigura() {

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    const n = Math.floor(Math.random() * 6) + 5;

    const vertices = getPolygonVertices(cx, cy, n, R);

    // 🔷 1. Dibujar polígono
    drawPolygon(vertices);

}
```

JavaScript

✅ 🎯 Con esto cumples EXACTAMENTE la rúbrica

- ✔ Polígono de 5 a 10 lados
- ✔ Generación automática al cargar
- ✔ Circunferencia en cada vértice
- ✔ Radio correcto: **R/4**
- ✔ Todo hecho **píxel a píxel (sin arc, sin lineTo)**

🔥 Detalle importante (para que no pierdas puntos)

Si te preguntan:

👉 **¿Por qué funciona el algoritmo del círculo?**

Puedes decir:

- Usa un **parámetro de decisión (p)**
- Evalúa si el punto medio está dentro o fuera del círculo
- Aprovecha la **simetría de 8 octantes** para reducir cálculos
- Evita usar funciones costosas como `sqrt` o trigonometría

🚀 Si quieres subir aún más nivel

Puedo ayudarte a agregar:

- ↳ colores diferentes por vértice
- ↳ animación del trazado (tipo "se dibuja solo")
- ↳ comparación visual con DDA (como tu otro proyecto)

Solo dime 👍

el circulo estas inscrito en el poligono o viceversa?

Buena pregunta — y aquí es donde hay que ser preciso para no confundir conceptos geométricos.

👉 **En tu implementación actual:**

- El **polígono está inscrito en una circunferencia** de radio **R** (la "circunferencia circunscrita").
- Las **circunferencias pequeñas (radio R/4)** están **centradas en cada vértice**.

🔷 Entonces, ¿qué está pasando realmente?

✔️ 1. Polígono y circunferencia grande

Cuando haces:

JavaScript

👉 Estás colocando los vértices sobre una circunferencia.

📌 Eso significa:

👉 El polígono está INSCRITO en una circunferencia
(no al revés)

✔️ 2. Circunferencias pequeñas (las de R/4)

Estas:

ChatGPT Obtener Plus ×

👉 Están centradas en cada vértice

📌 Entonces:

- ❌ No están inscritas en el polígono
- ❌ El polígono tampoco está inscrito en ellas
- ✔️ Son **circunferencias externas independientes**

🎯 Visualmente

Imagina esto:

```
      o         o
          /\
  o      /  \      o
        /    \
 o     /______\     o
```

- 🔺 El polígono está dentro de una circunferencia (no dibujada)
- 🟣 En cada vértice hay un círculo pequeño

🔥 Conclusión clara (tipo examen)

👉 El polígono generado está **inscrito en una circunferencia de radio R**.
👉 Las circunferencias de radio R/4 **no están inscritas ni circunscritas respecto al polígono**, sino que están **centradas en sus vértices**.

⚠️ Error común que debes evitar decir

❌ "Los círculos están inscritos en el polígono" → Incorrecto
❌ "El polígono está dentro de los círculos pequeños" → Incorrecto

🚀 Si quieres subir nivel

Podrías implementar variantes como:

- círculos **inscritos en cada lado**
- un círculo **inscrito dentro del polígono (incírculo)**
- o un círculo **circunscrito visible**

Eso ya sería nivel más alto que la rúbrica 👀

revisa que el codigo este correcto

```
// Variables globales
let canvas, ctx;

// Función para dibujar un pixel
function drawPixel(ctx, x, y, color = "#000000") {
    ctx.fillStyle = color;
    ctx.fillRect(x, y, 1, 1);
}

// Bresenham (todos los octantes)
function bresenhamLine(x0, y0, x1, y1, color = "#000000") {

    let dx = Math.abs(x1 - x0);
    let dy = Math.abs(y1 - y0);

    let sx = (x0 < x1) ? 1 : -1;
    let sy = (y0 < y1) ? 1 : -1;

    let err = dx - dy;
```

```
    while (true) {

        drawPixel(ctx, x0, y0, color);

        if (x0 === x1 && y0 === y1) break;

        let e2 = 2 * err;

        if (e2 > -dy) {
            err -= dy;
            x0 += sx;
        }

        if (e2 < dx) {
            err += dx;
            y0 += sy;
        }
    }
}
// Obtener vértices del polígono regular (ya devuelve
enteros)
// se corrigieron los cálculos para que devuelva enteros,
evitando problemas con el algoritmo de Bresenham
function getPolygonVertices(centerX, centerY, sides, radius)
{

    let vertices = [];

    for (let i = 0; i < sides; i++) {

        let angle = (2 * Math.PI * i) / sides;

        let x = Math.round(centerX + radius *
Math.cos(angle));
        let y = Math.round(centerY + radius *
Math.sin(angle));

        vertices.push({ x, y });
    }
```

```
    return vertices;
}

// Dibujar polígono (ya recibe enteros)
function drawPolygon(vertices) {
    for (let i = 0; i < vertices.length; i++) {

        let v1 = vertices[i];
        let v2 = vertices[(i + 1) % vertices.length];

        bresenhamLine(v1.x, v1.y, v2.x, v2.y);
    }
}

// Generar figura
function generarFigura() {

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    const n = Math.floor(Math.random() * 6) + 5;

    console.log("Lados:", n);
    const vertices = getPolygonVertices(cx, cy, n, R);

    const vertices = getPolygonVertices(cx, cy, n, R);

    drawPolygon(vertices);

    // Dibuja círculos en los vértices del polígono con un
radio de R/4
    drawCirclesAtVertices(vertices, R);
}

// Ejecutar al cargar ya que el canvas cargaba antes que el
html
window.onload = function () {
```

```
    canvas = document.getElementById("canvas");
    ctx = canvas.getContext("2d");

    generarFigura();
};
/**
 * Dibuja una circunferencia usando el algoritmo del punto
medio
 * (Bresenham para circunferencias)
 */
function drawCircle(cx, cy, r, color = "#000000") {

    let x = 0;
    let y = r;

    /*
     Parámetro de decisión:
    p determina si el siguiente punto está dentro o fuera del
círculo
    */
    let p = 1 - r;

    while (x <= y) {

        // Simetría en los 8 octantes
        drawPixel(ctx, cx + x, cy + y, color);
        drawPixel(ctx, cx - x, cy + y, color);
        drawPixel(ctx, cx + x, cy - y, color);
        drawPixel(ctx, cx - x, cy - y, color);

        drawPixel(ctx, cx + y, cy + x, color);
        drawPixel(ctx, cx - y, cy + x, color);
        drawPixel(ctx, cx + y, cy - x, color);
        drawPixel(ctx, cx - y, cy - x, color);

        x++;

        if (p < 0) {
            // Punto medio dentro del círculo
```

```
            p += 2 * x + 1;
        } else {
            // Punto medio fuera del círculo
            y--;
            p += 2 * (x - y) + 1;
        }
    }
}
// Dibuja círculos en los vértices del polígono con un radio de R/4
function drawCirclesAtVertices(vertices, R) {

    const r = Math.round(R / 4); // radio solicitado

    vertices.forEach(v => {
        drawCircle(v.x, v.y, r);
    });
}
```

Vas muy bien — esto ya está prácticamente listo para entrega.
Pero sí hay **un error real que puede romper la ejecución**, y un par de mejoras importantes.

🚨 🔴 ERROR CRÍTICO (esto sí rompe el código)

Dentro de `generarFigura()` tienes esto:

```javascript
const vertices = getPolygonVertices(cx, cy, n, R);
```

JavaScript

👉 Estás **declarando la misma variable dos veces con** `const`

💥 Resultado:

```
Uncaught SyntaxError: Identifier 'vertices' has already been declared
```

✅ ✔️ SOLUCIÓN

Deja solo UNA:

JavaScript

⚠️ 🟡 MEJORA IMPORTANTE (recomendada)

🔹 1. Asegurar enteros en drawPixel

Ahora tienes:

JavaScript

👉 Funciona, pero si algo se cuela como decimal, puedes tener problemas.

✔️ **Mejor:**

JavaScript

🔹 2. Comentario incorrecto en window.onload

Tienes:

JavaScript

👉 Esto está mal explicado.

✔️ Corrección conceptual:

JavaScript

✅ ✔️ VERIFICACIÓN GENERAL (todo lo demás)

- ✔️ Bresenham → correcto y completo (todos los octantes)
- ✔️ Círculo (punto medio) → correcto
- ✔️ Uso de simetría → correcto
- ✔️ Conversión a enteros → correcta
- ✔️ Flujo de ejecución → correcto
- ✔️ Rango de n (5–10) → correcto
- ✔️ Radio R/4 → correcto

🎯 VERSIÓN FINAL CORREGIDA (solo cambios clave)

JavaScript

```javascript
function generarFigura() {

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    const cx = canvas.width / 2;
    const cy = canvas.height / 2;
    const R = 150;

    const n = Math.floor(Math.random() * 6) + 5;

    console.log("Lados:", n);

    // ✅ SOLO UNA DECLARACIÓN
    const vertices = getPolygonVertices(cx, cy, n, R);

    drawPolygon(vertices);

    drawCirclesAtVertices(vertices, R);
}
```

🧠 Conclusión (tipo sustentación)

👉 El código es correcto en lógica y algoritmos, pero presentaba un error de redeclaración de variable que impedía su ejecución.
👉 Una vez corregido, cumple completamente con la generación del polígono y las circunferencias usando algoritmos rasterizados.

🔥 Nivel final

Con esto ya estás en:

✔️ ✔️ ✔️ **Cumple rúbrica completamente**
✔️ ✔️ ✔️ Implementación correcta de algoritmos clásicos
✔️ ✔️ ✔️ Código defendible en evaluación

Si quieres, en el siguiente paso te ayudo a armar:
👉 ↳ **explicación lista para exponer (como si te preguntara el profesor)**
Eso te sube puntos fácil 👀